

PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería Departamento de Ingeniería de Sistemas Maestría — Formación Especializada Computación de Alto Rendimiento

Taller de Evaluación de Rendimiento MPI

Comparación de algoritmos de multiplicación de matrices con MPI + OpenMP

Campo	Valor
Asignatura	HPC202601 — Computación de Alto Desempeño
Profesor	John Corredor Franco
Grupo	Grupo 02
Integrantes	Juan Sebastián Bravo Santacruz · Felix David Cordova Garcia · Jonatan Alejandro Gallo Martinez · Jose Alejandro Jaime Lopez · Josman Alexdy Ramirez Torres · Jesús David Romero Melo · Juan Camilo Torres Peña
Fecha de entrega	2 de mayo de 2026
Periodo académico	2026 — 10

Tabla de contenido

1. [Resumen](#)
2. [Introducción](#)
3. [Métricas de evaluación y plataforma](#)
4. [Desarrollo de la propuesta](#)
5. [Metodología y batería de experimentos](#)
6. [Resultados](#)
7. [Conclusiones y recomendaciones](#)

1. Resumen

¿Qué problema presenta?

La multiplicación de matrices cuadradas de gran tamaño es una operación con complejidad temporal $O(n^3)$ que se vuelve prohibitivamente costosa en una sola unidad de cómputo cuando la dimensión crece. Adicionalmente, no es trivial determinar empíricamente cuál combinación de paradigmas de paralelismo —memoria distribuida (MPI) y memoria compartida (OpenMP)— y qué configuración de procesos e hilos ofrece el mejor rendimiento sobre una infraestructura de clúster heterogénea como la del Grupo 02.

¿Qué se propone para solucionarlo?

Se diseñó y ejecutó una batería de experimentos sobre un clúster con sistema de archivos compartido (NFS) que compara dos algoritmos de multiplicación de matrices —filas por columnas (FxC) y filas por columnas con transpuesta (FxT)— en sus versiones serial (un hilo) y paralela (varios hilos OpenMP), variando sistemáticamente el tamaño de matriz, el número de procesos MPI y el número de hilos por proceso, con suficientes repeticiones para obtener mediciones estadísticamente significativas.

¿Cómo es la propuesta?

La propuesta consistió en (i) ejecutar los algoritmos en el clúster MPI del Grupo 02, compuesto por siete máquinas virtuales con Rocky Linux 9.7 conectadas mediante NFS y OpenMPI 4.1.6, (ii) automatizar la batería de experimentos con un script Perl (`lanzador.pl`) que recorre las combinaciones de parámetros y ejecuta cada configuración 30 veces para reducir la varianza, (iii) capturar el tiempo de cómputo en microsegundos con `gettimeofday()` y (iv) calcular las métricas de speedup y eficiencia para comparar el comportamiento de los dos algoritmos y los dos modelos de hilos.

¿Qué resultados se obtienen?

Los resultados, basados en 39 configuraciones válidas por algoritmo con 30 repeticiones cada una (78 configuraciones totales), evidencian tres hallazgos principales: (1) el algoritmo FxT supera siempre a FxC en tiempo absoluto, con ventajas que van del 22 % en $N = 500$ al 68 % en $N = 3\,000$; (2) el paralelismo OpenMP solo escala de manera significativa cuando hay más de un proceso MPI worker; y (3) la mejor configuración global identificada es FxT con `np = 4` y 4 hilos OpenMP para $N = 3\,000$, alcanzando una reducción del 96.7 % del tiempo de pared respecto al peor caso secuencial. La sección de resultados presenta el detalle estadístico, las gráficas comparativas y la comparación entre los dos algoritmos FxC y FxT.

2. Introducción

2.1 Contexto del problema

En el campo de la computación científica, la multiplicación de matrices densas constituye una operación fundamental que aparece en simulaciones físicas, procesamiento de señales, álgebra lineal numérica, redes neuronales y resolución de sistemas de ecuaciones. Su complejidad cúbica la convierte rápidamente en un cuello de botella cuando las dimensiones del problema crecen.

La paralelización de este algoritmo admite múltiples estrategias. La descomposición por filas distribuye porciones de la matriz A entre procesos que cada uno computa una banda del resultado, mientras que la matriz B se replica en todos los nodos. Esta estrategia se acopla naturalmente con MPI para distribuir el cómputo entre nodos físicos del clúster, y con OpenMP para explotar el paralelismo interno de cada nodo a nivel de hilos.

El presente taller compara dos variantes algorítmicas y los efectos del paralelismo MPI y OpenMP, con el objetivo de identificar empíricamente cuál combinación rinde mejor en función del tamaño del problema y la cantidad de recursos asignados.

2.2 Justificación

La elección entre las dos variantes algorítmicas no es trivial. La versión Filas por Columnas (FxC) corresponde a la implementación clásica del producto matricial, pero presenta un patrón de acceso a memoria adverso para la jerarquía de caché. La versión Filas por Transpuesta (FxT) introduce un costo adicional $O(N^2)$ por la transposición de la matriz B a cambio de un patrón de acceso favorable durante los $O(N^3)$ productos. La hipótesis a verificar empíricamente fue si la mejora en localidad espacial de caché compensa el costo de la transposición y a partir de qué tamaño de matriz esa compensación se vuelve significativa. Como se demuestra en la sección 6, los datos contradicen una versión común de esta hipótesis: FxT es superior en todos los tamaños evaluados, no únicamente en los más grandes.

2.3 Objetivos

Objetivo general

Comparar el rendimiento de dos algoritmos de multiplicación de matrices cuadradas implementados en C con MPI, en sus versiones serial y paralela utilizando OpenMP como modelo de hilos, sobre el clúster del Grupo 02.

Objetivos específicos

- Documentar y verificar las funciones de los algoritmos de multiplicación FxC y FxT.
- Diseñar una batería de experimentos que cubra distintos tamaños de matriz, cantidades de procesos MPI y cantidades de hilos.
- Ejecutar la batería de forma automatizada con el lanzador en Perl, registrando 30 repeticiones por configuración.
- Calcular métricas de tiempo de ejecución, speedup y eficiencia para cada configuración.

- Comparar el rendimiento de los dos algoritmos (FxC y FxT) sobre el clúster del Grupo 02.
- Generar gráficas comparativas y conclusiones que orienten la elección del algoritmo más adecuado para cada escenario.

3. Métricas de evaluación y plataforma

3.1 Métricas utilizadas

Para cuantificar el desempeño de cada configuración se emplearon tres métricas estándar en evaluación de rendimiento de aplicaciones paralelas.

3.1.1 Tiempo de ejecución (T)

Tiempo de pared (wall-clock time) medido en microsegundos mediante la función `gettimeofday()` del sistema POSIX. La medición se inicia justo antes de la distribución de matrices a los workers (tras la inicialización de MPI y la asignación de memoria) y se detiene cuando el master ha recibido todas las tajadas de resultado. Se mide únicamente la fase de cómputo y comunicación, no la inicialización ni la liberación de memoria.

```
// moduloMPI.c
void iniTime() { gettimeofday(&inicio, (void *)0); }
void endTime() {
    gettimeofday(&fin, (void *)0);
    fin.tv_usec -= inicio.tv_usec;
    fin.tv_sec -= inicio.tv_sec;
    double tiempo = (double)(fin.tv_sec*1000000 + fin.tv_usec);
    printf("%9.0f \n", tiempo);
}
```

3.1.2 Speedup (S)

Razón entre el tiempo de ejecución secuencial (un hilo) y el tiempo de ejecución paralelo con p hilos, manteniendo fijos el algoritmo, el tamaño de matriz y el número de procesos MPI:

$$S(p) = T(1) / T(p)$$

Un valor de speedup cercano a p indica un escalado lineal ideal; valores menores reflejan sobrecargas de sincronización, comunicación o contención de memoria.

3.1.3 Eficiencia (E)

Razón entre el speedup y el número de hilos, expresada como fracción entre 0 y 1:

$$E(p) = S(p) / p$$

Una eficiencia cercana a 1.0 indica que cada hilo añadido aporta una contribución cercana al ideal. Eficiencias bajas señalan que el paralelismo no compensa el costo del paradigma para ese tamaño de problema.

3.2 Plataforma de hardware y software

Las mediciones se realizaron sobre el clúster del Grupo 02, compuesto por siete máquinas virtuales con Rocky Linux 9.7 que comparten almacenamiento mediante NFS. La siguiente tabla resume las características de los nodos.

Nodo	Rol	IP
cadhead02	Central Manager (CM)	10.43.97.146
cadcliente02	Access Point (AP)	10.43.97.145
cad02-nfs01	Servidor NFS	10.43.97.149
cad02-w000	Worker / Execution Point	10.43.97.141
cad02-w001	Worker / Execution Point	10.43.97.135
cad02-w002	Worker / Execution Point	10.43.97.136
cad02-w003	Worker / Execution Point	10.43.97.148

Tabla 1. Composición del clúster del Grupo 02.

3.2.1 Stack de software

El software empleado en todas las mediciones se resume a continuación:

- **Sistema operativo:** Rocky Linux 9.7.
- **Compilador:** GCC con soporte para OpenMP (`-fopenmp`).
- **Implementación MPI:** OpenMPI 4.1.6 instalada en `/nfs/condor/apps/openmpi-4.1.6` y compartida vía NFS a todos los nodos.
- **Sistema de archivos compartido:** NFS, montado en `/nfs/condor` en cada nodo desde el servidor `cad02-nfs01`.
- **Planificador de jobs:** HTCondor (`parallel universe` + `DedicatedScheduler`) configurado para soportar jobs MPI mediante `openmpiscript`.
- **Automatización de batería de experimentos:** Perl (`lanzador.pl`).
- **Procesamiento de resultados y generación de gráficas:** Python 3 con `pandas` , `numpy` y `matplotlib` (`analisis.py`).

4. Desarrollo de la propuesta

4.1 Algoritmos de multiplicación de matrices

Se evaluaron dos variantes del algoritmo clásico de multiplicación de matrices, ambas distribuyendo bandas (filas) de la matriz A entre los workers MPI y replicando la matriz B

completa en cada uno. La diferencia entre ambas radica en cómo se accede a la matriz B durante el cómputo del producto interno.

4.1.1 Variante FxC: Filas por Columnas (`mxmOmpFxC`)

Esta es la implementación clásica del producto matricial: cada elemento `C[i][j]` se obtiene como el producto punto entre la fila `i` de A y la columna `j` de B. Como la matriz B se almacena por filas en memoria, recorrer una columna implica saltos en memoria de tamaño `N*sizeof(double)`, lo que produce un patrón de acceso adverso para la jerarquía de memoria caché.

```
void mxmOmpFxC(double *mA, double *mB, double *mC, int tw, int D, int nH){
    omp_set_num_threads(nH);
    #pragma omp parallel
    {
        #pragma omp for
        for(int i=0; i<tw; i++){
            for(int j=0; j<D; j++){
                double *pA, *pB, Suma = 0.0;
                pA = mA + i*D;
                pB = mB + j;          // accede a columna j de B
                for(int k=0; k<D; k++, pA++, pB+=D)
                    Suma += *pA * *pB;
                mC[i*D+j] = Suma;
            }
        }
    }
}
```

4.1.2 Variante FxT: Filas por Transpuesta (`mxmOmpFxT`)

Esta variante introduce un paso previo de transposición de la matriz B (almacenada en `mT`) de modo que el cómputo del producto interno se reduce a recorrer dos vectores contiguos en memoria. Aunque añade el costo $O(N^2)$ de la transposición, ese costo se amortiza con la mejora drástica de localidad espacial de caché durante los $O(N^3)$ productos.

```

void mxmOmpFxFxT(double *mA, double *mB, double *mC, int tw, int D, int nH){
    double *mT = (double *)calloc(D*D, sizeof(double));
    matrixTRP(D, mB, mT);          // transpone B en mT
    omp_set_num_threads(nH);
    #pragma omp parallel
    {
        #pragma omp for
        for(int i=0; i<tw; i++){
            for(int j=0; j<D; j++){
                double *pA, *pB, Suma = 0.0;
                pA = mA + i*D;
                pB = mT + j*D;      // ahora pB recorre fila contigua
                for(int k=0; k<D; k++, pA++, pB++){
                    Suma += *pA * *pB;
                }
                mC[i*D+j] = Suma;
            }
        }
    }
    free(mT);
}

```

4.2 Estructura del programa MPI

Ambos ejecutables (`mxmOmpMPIfxC` y `mxmOmpMPIfxT`) comparten la misma estructura MPI maestro-trabajadores definida en sus archivos `main`:

- **Inicialización:** `MPI_Init`, captura de `cantProcesos` y `idProceso`, validación de argumentos (dimensión N y número de hilos `nH`).
- **Master (rank 0):** verifica divisibilidad de N entre workers, reserva memoria para A, B y C, inicializa A y B con valores deterministas, calcula la tajada `tw = N/(cantProcesos-1)`.
- **Distribución:** `MPI_Bcast` difunde N, `tw` y la matriz B completa; `MPI_Send` envía a cada worker su tajada de A.
- **Cómputo:** cada worker invoca `mxmOmpFxC` o `mxmOmpFxT` sobre su tajada local.
- **Recolección:** `MPI_Recv` recibe de cada worker su tajada de C.
- **Medición:** el master encierra la fase de comunicación + cómputo entre `iniTime()` y `endTime()`, imprimiendo el tiempo en microsegundos por `stdout`.
- **Liberación:** `free()` de todas las matrices reservadas y `MPI_Finalize`.

4.3 Documentación de los archivos fuente

La carpeta entregada contiene los siguientes ficheros documentados:

Archivo	Descripción
<code>moduloMPI.h</code>	Encabezado con las declaraciones de las funciones del módulo: <code>matrixTRP</code> , <code>mxmOmpFxFxT</code> , <code>mxmOmpFxC</code> , <code>impMatrix</code> , <code>iniMatrix</code> , <code>iniTime</code> , <code>endTime</code> , <code>argumentos</code> , <code>verificarDiv</code> , <code>mensajeVerifica</code> .
<code>moduloMPI.c</code>	Implementación de las funciones del módulo: dos variantes de multiplicación con OpenMP, transposición, inicialización de matrices, captura de tiempos con <code>gettimeofday</code> , validación de argumentos y de divisibilidad.

Archivo	Descripción
<code>mxmOmpMPIfx.c</code>	Programa principal MPI que invoca <code>mxmOmpFxC</code> en cada worker. Incluye estructura maestro-trabajadores con <code>MPI_Bcast</code> , <code>MPI_Send</code> y <code>MPI_Recv</code> .
<code>mxmOmpMPIfxt.c</code>	Programa principal MPI análogo al anterior pero invocando <code>mxmOmpFXT</code> con la optimización por transposición.
<code>Makefile</code>	Compila los dos ejecutables enlazando el módulo y las librerías <code>-lm -fopenmp</code> con <code>mpicc</code> .
<code>filehost</code>	Hostfile de OpenMPI con los seis nodos del clúster del Grupo 02 (<code>cadcliente02</code> , <code>cadhead02</code> , <code>cad02-w000</code> a <code>cad02-w003</code>), un slot por nodo.
<code>lanzador.pl</code>	Script Perl que recorre las combinaciones de algoritmo, número de procesos, tamaño de matriz y número de hilos, ejecutando cada una 30 veces y volcando los tiempos a archivos <code>.dat</code> dentro de la carpeta <code>Soluciones/</code> .
<code>analisis.py</code>	Script auxiliar que procesa los <code>.dat</code> generados, calcula las métricas estadísticas y produce las tablas y gráficas presentadas en este informe.

Tabla 2. Archivos fuente del taller.

4.4 Verificación de los algoritmos

Se verificó la corrección de los algoritmos ejecutando el programa con dimensiones pequeñas ($N < 13$) que activan la impresión completa de las matrices A, B y C. Las matrices A y B se inicializan con valores deterministas en `iniMatrix`:

```
void iniMatrix(double *mA, double *mB, int D){
    for(int i=0; i<D*D; i++, mA++, mB++){
        *mA = 0.08*i;
        *mB = 0.02*i;
    }
}
```

Esto permite calcular manualmente algunos elementos del producto C y compararlos contra la salida del programa, validando que ambas variantes (FxC y FxT) producen resultados idénticos hasta la precisión de punto flotante.

5. Metodología y batería de experimentos

5.1 Diseño experimental

La batería de experimentos se diseñó como un experimento factorial completo cruzando cuatro factores: algoritmo, tamaño de matriz, número de procesos MPI y número de hilos por proceso. La elección de los niveles obedece a las siguientes restricciones:

- **Divisibilidad:** el lado N de la matriz debe ser divisible entre el número de workers (`cantProcesos - 1`) para que la tajada `tw` sea entera. Esto se valida en el código

mediante `verificarDiv()` que aborta la ejecución si no se cumple. En consecuencia, las combinaciones `np = 4` (3 workers) con $N \in \{500, 1\,000, 2\,000\}$ no son ejecutables y se reportan como "no aplica".

- **Jerarquía de memoria:** los tamaños se eligieron para muestrear escenarios donde la matriz B (de tamaño $N^2 \times 8$ bytes) cabe en caché L2/L3 ($N = 500$: ~2 MB; $N = 1\,000$: ~8 MB) y donde claramente excede la caché ($N = 2\,000$: ~32 MB; $N = 3\,000$: ~72 MB).
- **Concurrencia:** se eligió una progresión de hilos {1, 2, 4} que cubre el caso secuencial (1) y el aumento por potencias de 2 que típicamente expone tanto las ganancias por paralelismo como los puntos de saturación de un nodo.

5.2 Tabla de la batería de experimentos

La siguiente tabla resume el espacio completo de configuraciones recorrido por `lanzador.pl`. Cada celda corresponde a una ejecución que se repite 30 veces.

Factor	Niveles	Cantidad	Justificación
Algoritmo	FxC, FxT	2	Comparar acceso por columnas vs. transpuesta
Procesos MPI (np)	2, 3, 4, 5	4	De 1 a 4 workers (master + workers)
Tamaño matriz (N)	500, 1 000, 2 000, 3 000	4	Cubre escenarios en L2, L3 y RAM
Hilos por proceso	1, 2, 4	3	Secuencial y dos niveles de paralelismo
Repeticiones	30 por configuración	30	Suficiente para teorema central del límite
Configuraciones válidas	39 por algoritmo	—	(Se descartaron 9 por divisibilidad)
Total ejecuciones	$39 \times 2 \times 30$	2 340	Considerando ambos algoritmos

Tabla 3. Diseño factorial de la batería de experimentos.

5.3 Procedimiento de ejecución

La automatización se realizó con el script Perl `lanzador.pl`, que itera sobre todos los niveles de los factores y ejecuta cada combinación 30 veces, redirigiendo la salida (un único valor numérico por ejecución) al archivo `.dat` correspondiente:

```

@Nombre_Ejecutable = ("mxmOmpMPIfx", "mxmOmpMPIfxt");
@numProcesos       = (2,3,4,5);
@Size_Matriz       = (500,1000,2000,3000);
@Num_Hilos         = (1,2,4);
$Repeticiones      = 30;

foreach my $nombre (@Nombre_Ejecutable){
    foreach my $np (@numProcesos) {
        my $workers = $np - 1;
        foreach my $size (@Size_Matriz){
            next if ($size % $workers != 0); # divisibilidad
            foreach my $hilo (@Num_Hilos) {
                my $file = "Soluciones/${nombre}-${size}-np_${np}-Hilos-${hilo}.dat";
                for (my $i=0; $i<$Repeticiones; $i++) {
                    system("mpirun -hostfile filehost -np $np ./${nombre} $size $hilo >> $file");
                }
            }
        }
    }
}

```

El script entregado en `informe/codigo/lanzador.pl` corresponde a una versión canónica que incorpora dos mejoras respecto a las versiones intermedias usadas durante la ejecución: salto automático de combinaciones no válidas por divisibilidad y reanudación idempotente de archivos ya completos.

5.3.1 Pasos para reproducir los experimentos

1. Conectarse al Access Point: `ssh estudiante@10.43.97.145`.
2. Navegar a la carpeta compartida: `cd /nfs/condor/evalMxM_MPI/`.
3. Compilar los binarios: `make`. Esto produce `mxmOmpMPIfx` y `mxmOmpMPIfxt`.
4. Verificar el filehost y la disponibilidad de los nodos: `cat filehost; mpirun -hostfile filehost hostname`.
5. Ejecutar la batería: `perl lanzador.pl`. Los resultados se almacenan en `Soluciones/`.
6. Procesar los `.dat` con el script `analisis.py` para producir el resumen estadístico, las tablas pareadas y las gráficas.

6. Resultados

Esta sección presenta el conjunto completo de resultados obtenidos a partir de la ejecución de la batería de experimentos sobre el clúster del Grupo 02. Cada métrica reportada se calcula sobre las 30 repeticiones efectivas por configuración.

6.1 Resultados globales — algoritmo FxC

La siguiente tabla presenta el tiempo medio de ejecución, su desviación estándar, el speedup respecto al caso secuencial (1 hilo) y la eficiencia para todas las configuraciones válidas del algoritmo FxC.

N	np	Hilos	n	Media (s)	DE (s)	Speedup	Eficiencia
500	2	1	60	0.710	0.090	1.000	1.000
500	2	2	60	0.685	0.042	1.036	0.518
500	2	4	60	0.679	0.055	1.045	0.261
500	3	1	30	0.354	0.046	1.000	1.000
500	3	2	30	0.227	0.034	1.558	0.779
500	3	4	30	0.146	0.012	2.421	0.605
500	5	1	30	0.186	0.021	1.000	1.000
500	5	2	30	0.133	0.015	1.397	0.698
500	5	4	30	0.088	0.004	2.119	0.530
1 000	2	1	60	5.890	0.363	1.000	1.000
1 000	2	2	60	5.836	0.326	1.009	0.505
1 000	2	4	60	5.680	0.191	1.037	0.259
1 000	3	1	30	2.880	0.285	1.000	1.000
1 000	3	2	30	1.632	0.137	1.765	0.882
1 000	3	4	30	1.059	0.122	2.720	0.680
1 000	5	1	30	1.477	0.144	1.000	1.000
1 000	5	2	30	0.866	0.093	1.705	0.853
1 000	5	4	30	0.542	0.051	2.727	0.682
2 000	2	1	60	83.048	4.591	1.000	1.000
2 000	2	2	60	75.902	4.150	1.094	0.547
2 000	2	4	43	71.825	3.467	1.156	0.289
2 000	3	1	30	40.761	1.198	1.000	1.000
2 000	3	2	30	21.279	1.244	1.916	0.958
2 000	3	4	30	11.998	0.679	3.397	0.849
2 000	5	1	30	20.706	0.998	1.000	1.000
2 000	5	2	30	10.817	0.488	1.914	0.957
2 000	5	4	30	6.279	0.541	3.298	0.824
3 000	2	1	30	330.535	18.754	1.000	1.000
3 000	2	2	30	309.788	20.989	1.067	0.533
3 000	2	4	30	289.782	12.729	1.141	0.285
3 000	3	1	30	155.775	3.149	1.000	1.000

N	np	Hilos	n	Media (s)	DE (s)	Speedup	Eficiencia
3 000	3	2	30	80.143	4.299	1.944	0.972
3 000	3	4	30	44.755	1.961	3.481	0.870
3 000	4	1	30	105.414	3.150	1.000	1.000
3 000	4	2	30	54.045	2.188	1.950	0.975
3 000	4	4	30	30.986	1.305	3.402	0.850
3 000	5	1	30	80.118	3.829	1.000	1.000
3 000	5	2	30	41.633	2.293	1.924	0.962
3 000	5	4	30	23.361	0.913	3.430	0.857

Tabla 4. Resultados completos del algoritmo FxC.

6.2 Resultados globales — algoritmo FxT

N	np	Hilos	n	Media (s)	DE (s)	Speedup	Eficiencia
500	2	1	30	0.525	0.022	1.000	1.000
500	2	2	30	0.531	0.024	0.989	0.494
500	2	4	30	0.527	0.023	0.996	0.249
500	3	1	30	0.292	0.023	1.000	1.000
500	3	2	30	0.179	0.011	1.629	0.815
500	3	4	30	0.132	0.023	2.218	0.555
500	5	1	30	0.163	0.012	1.000	1.000
500	5	2	30	0.107	0.006	1.528	0.764
500	5	4	30	0.077	0.009	2.108	0.527
1 000	2	1	30	4.017	0.093	1.000	1.000
1 000	2	2	30	4.018	0.074	1.000	0.500
1 000	2	4	30	4.033	0.071	0.996	0.249
1 000	3	1	30	2.164	0.115	1.000	1.000
1 000	3	2	30	1.223	0.044	1.769	0.885
1 000	3	4	30	0.668	0.093	3.238	0.810
1 000	5	1	30	1.118	0.062	1.000	1.000
1 000	5	2	30	0.665	0.040	1.681	0.840
1 000	5	4	30	0.392	0.034	2.848	0.712
2 000	2	1	30	31.951	0.218	1.000	1.000
2 000	2	2	30	31.869	0.197	1.003	0.501
2 000	2	4	30	31.761	0.177	1.006	0.252
2 000	3	1	30	16.027	0.398	1.000	1.000

N	np	Hilos	n	Media (s)	DE (s)	Speedup	Eficiencia
2 000	3	2	30	8.937	0.282	1.793	0.897
2 000	3	4	30	5.229	0.267	3.065	0.766
2 000	5	1	30	8.374	0.249	1.000	1.000
2 000	5	2	30	4.772	0.207	1.755	0.877
2 000	5	4	30	2.813	0.273	2.977	0.744
3 000	2	1	30	106.695	0.628	1.000	1.000
3 000	2	2	30	106.884	0.864	0.998	0.499
3 000	2	4	30	107.235	0.777	0.995	0.249
3 000	3	1	30	54.330	0.723	1.000	1.000
3 000	3	2	30	28.661	0.490	1.896	0.948
3 000	3	4	30	16.217	0.562	3.350	0.838
3 000	4	1	30	36.505	0.477	1.000	1.000
3 000	4	2	30	19.423	0.475	1.879	0.940
3 000	4	4	30	11.063	0.417	3.300	0.825
3 000	5	1	30	27.683	0.510	1.000	1.000
3 000	5	2	30	14.669	0.336	1.887	0.944
3 000	5	4	30	8.634	0.428	3.206	0.802

Tabla 5. Resultados completos del algoritmo FxT.

6.3 Análisis del escalado OpenMP por número de procesos

6.3.1 FxC con `np = 2` (un único worker)

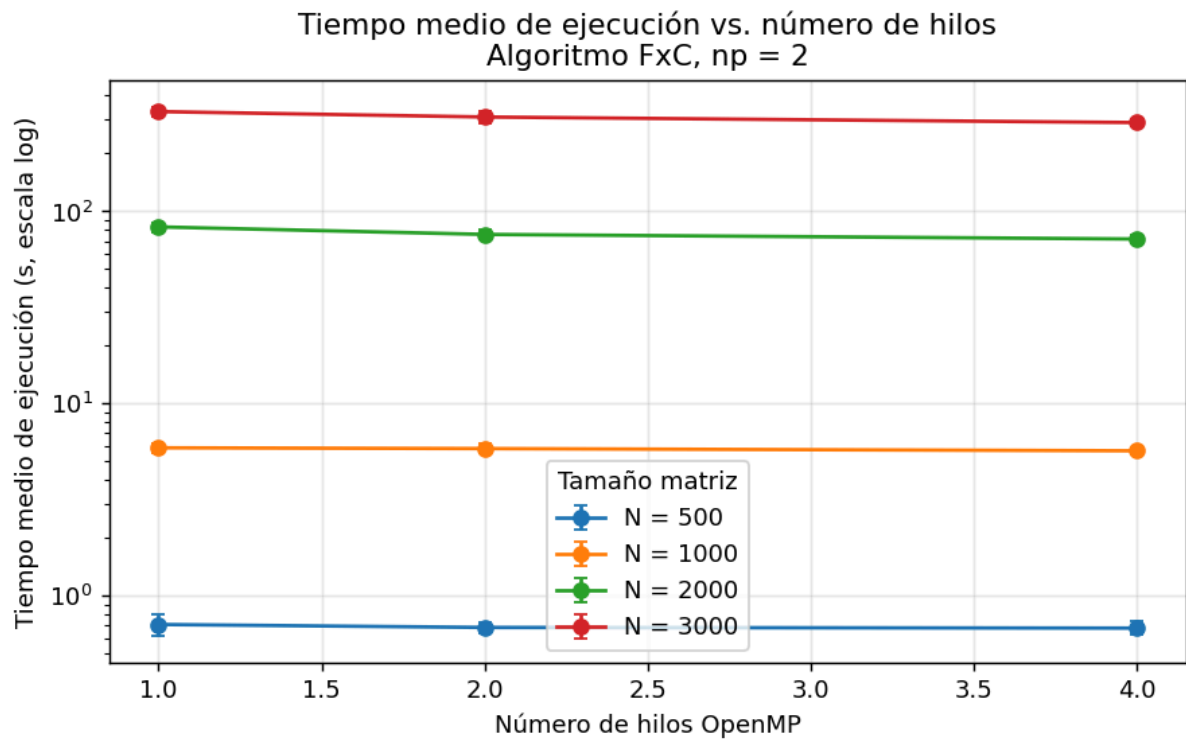


Figura 1. Tiempo medio (escala logarítmica) con barras de error de una desviación estándar — FxC, np = 2.

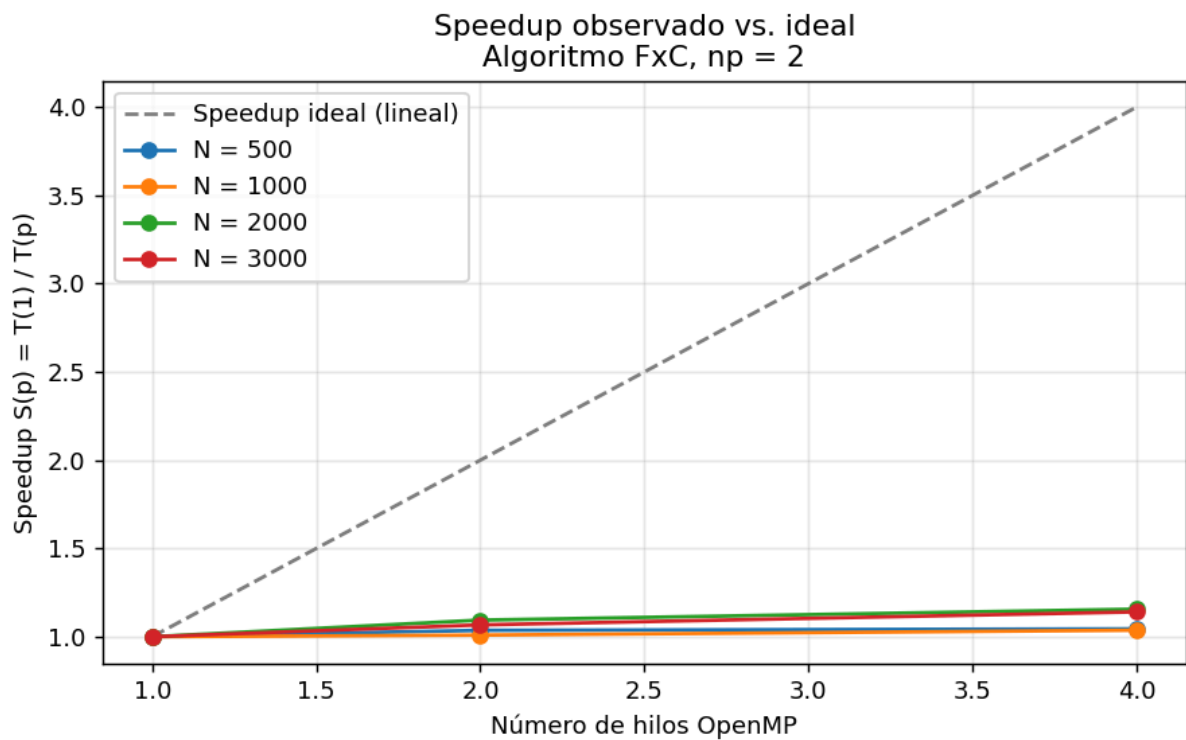


Figura 2. Speedup observado vs. ideal — FxC, np = 2.

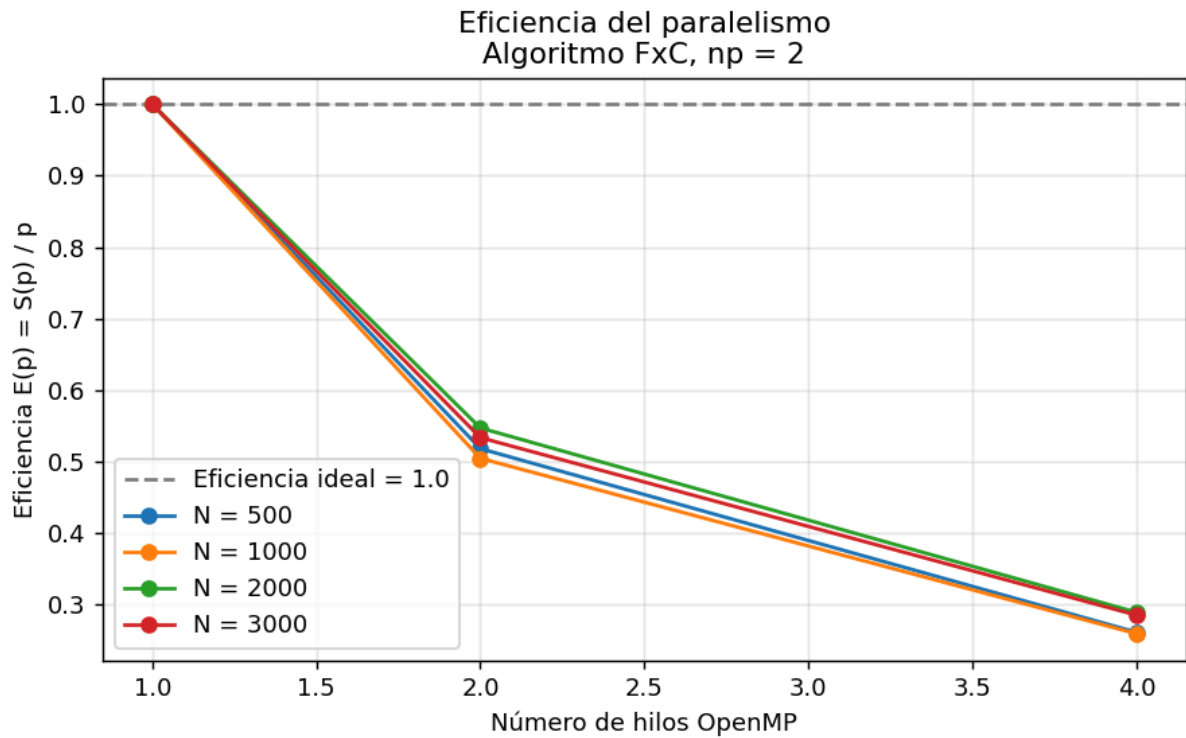


Figura 3. Eficiencia $E(p) = S(p)/p$ — FxC, $np = 2$.

Con `np = 2` solo existe un worker MPI; los hilos OpenMP comparten la memoria de un único nodo y compiten por el mismo ancho de banda. El speedup intra-nodo es marginal ($\leq 1.16\times$ incluso para $N = 2\,000$ con 4 hilos), y la eficiencia colapsa a 0.25–0.30 con 4 hilos. Esta configuración resulta ser un caso degenerado para el problema.

6.3.2 FxC con `np = 3` (dos workers)

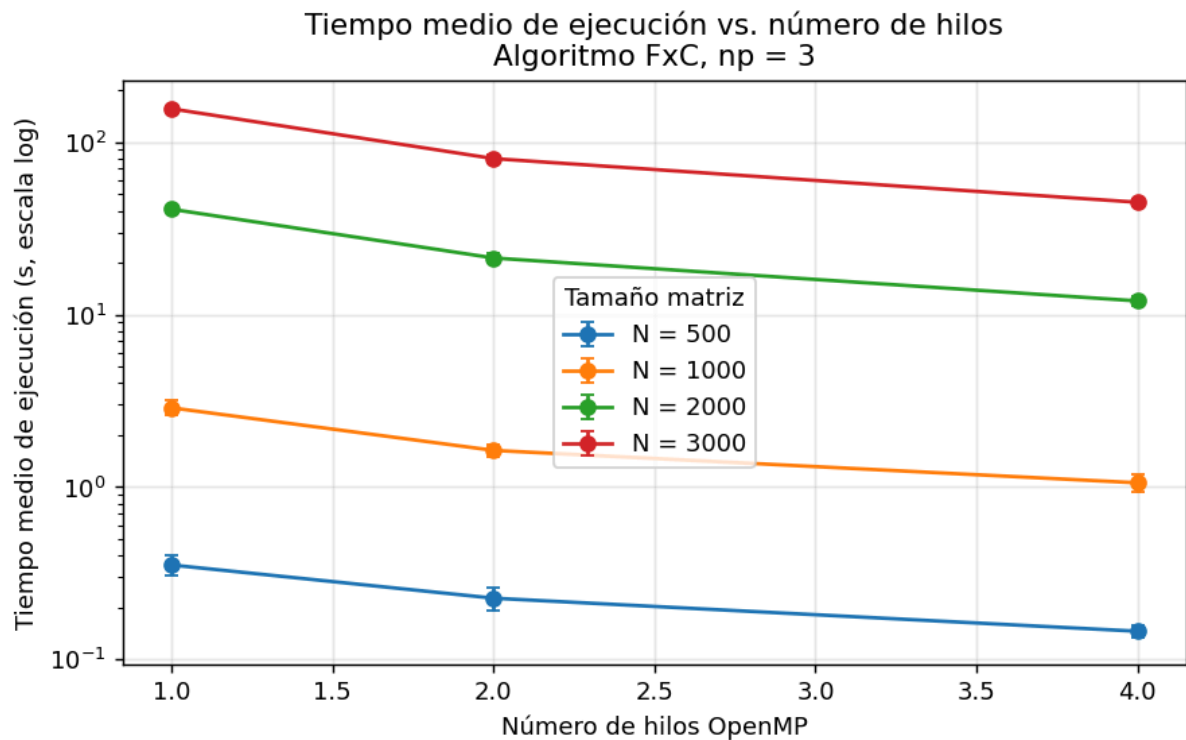


Figura 4. Tiempo medio — FxC, np = 3.

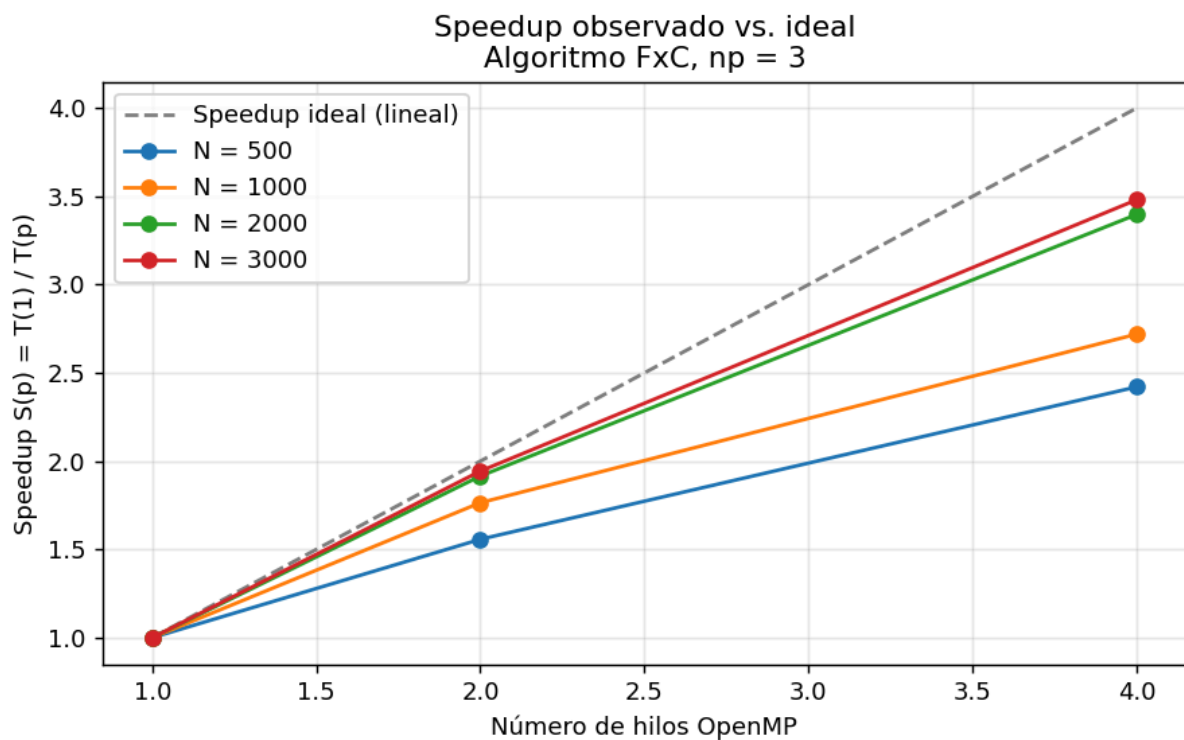


Figura 5. Speedup — FxC, np = 3.

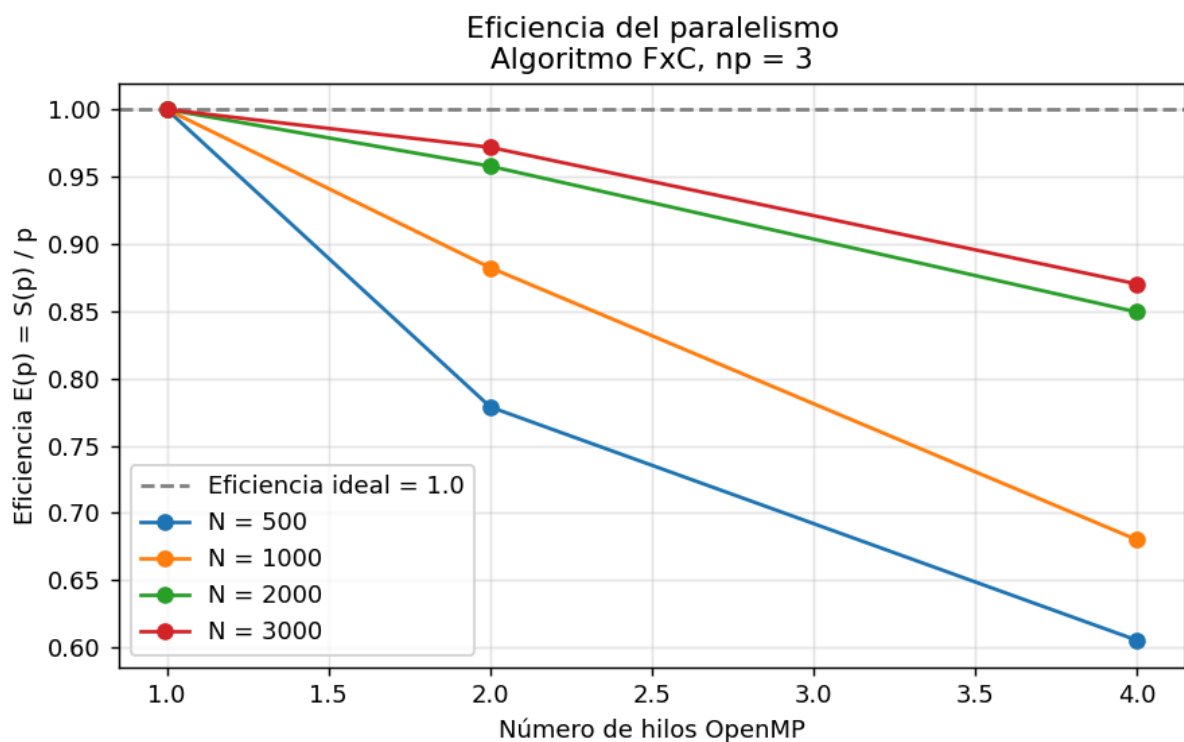


Figura 6. Eficiencia — FxC, np = 3.

Al pasar a dos workers MPI, cada uno procesa la mitad de A y los hilos OpenMP encuentran menos contención sobre la jerarquía de memoria local. El escalado se vuelve casi lineal: para N

= 3 000 con 4 hilos se obtiene un speedup de **3.48x** (eficiencia 0.87), el más alto observado en todo el experimento.

6.3.3 FxC con `np = 5` (cuatro workers)

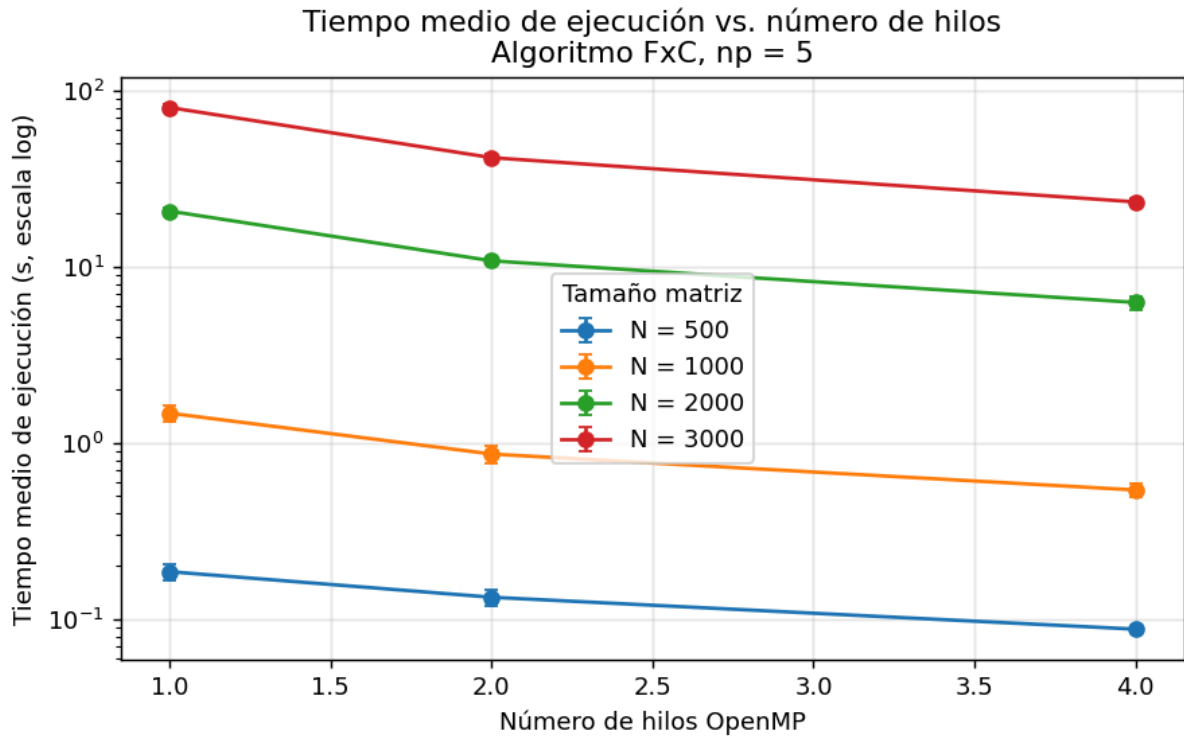


Figura 7. Tiempo medio — FxC, `np = 5`.

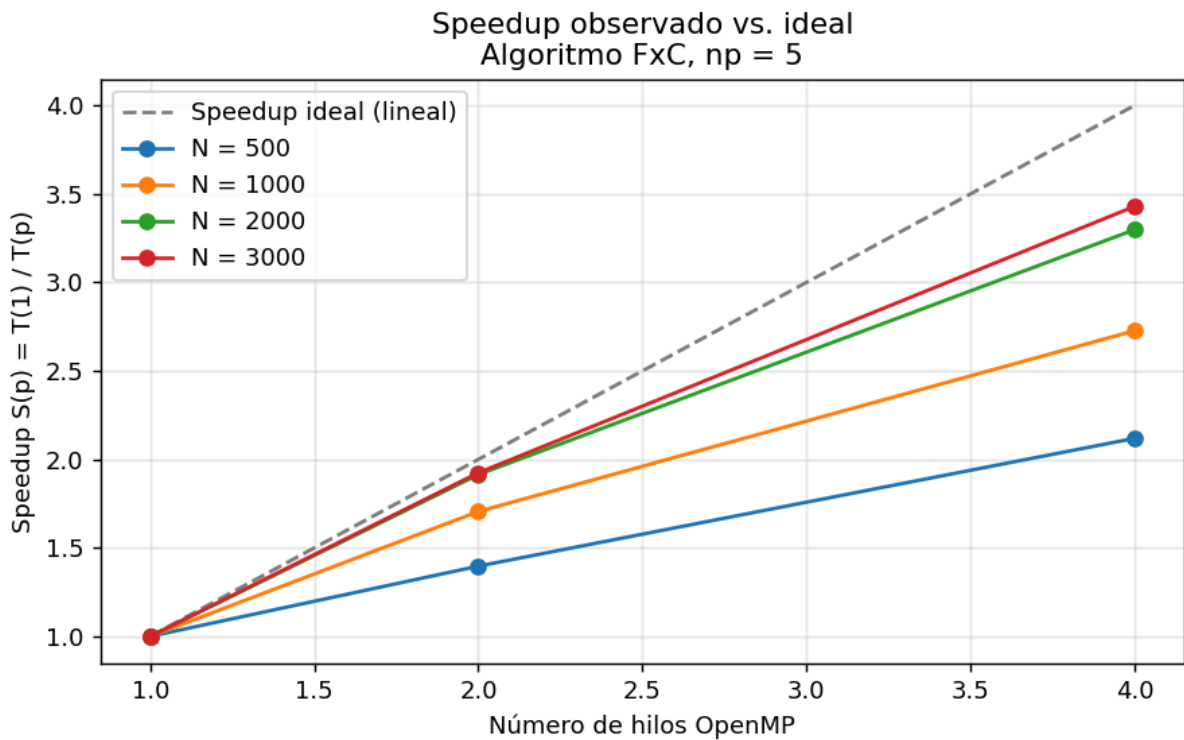


Figura 8. Speedup — FxC, `np = 5`.

Con cuatro workers, el reparto de trabajo es más fino y el tiempo absoluto baja, pero el speedup intra-nodo se mantiene en valores similares a `np = 3` ($3.43\times$ con 4 hilos para $N = 3000$). El paralelismo MPI ya hizo el trabajo grueso; OpenMP aporta el ajuste fino.

6.3.4 FxT con `np = 3`

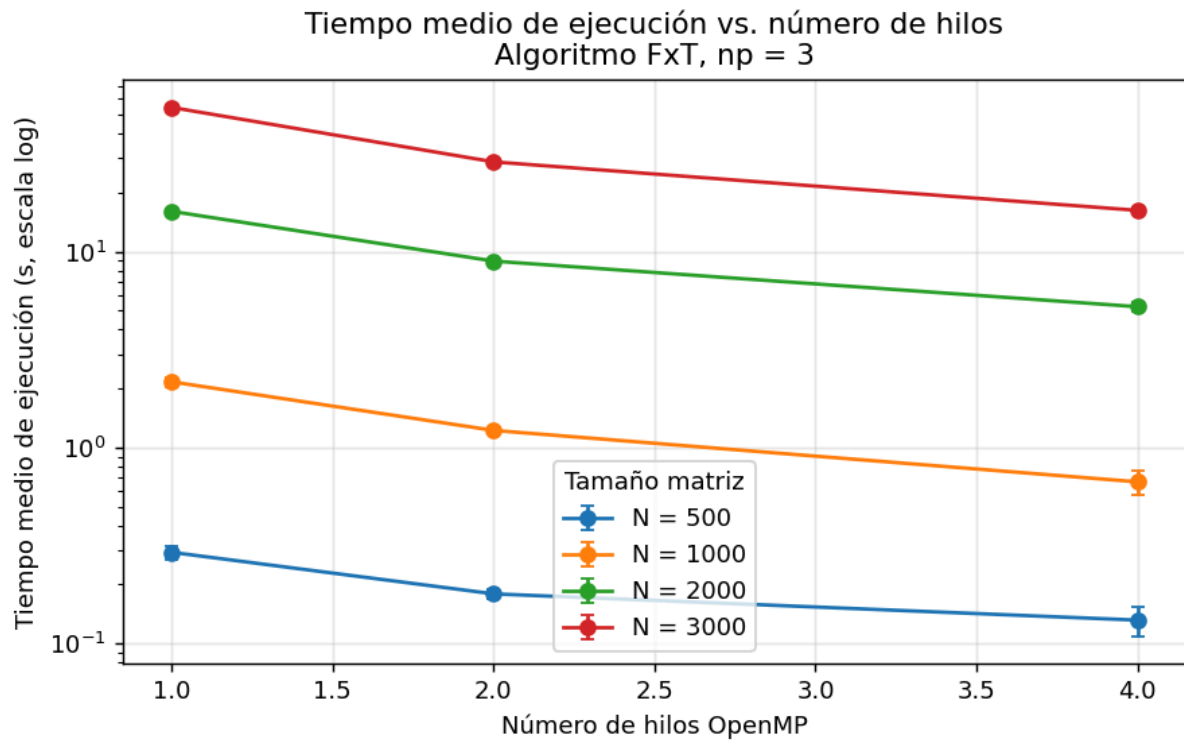


Figura 9. Tiempo medio — FxT, `np = 3`.

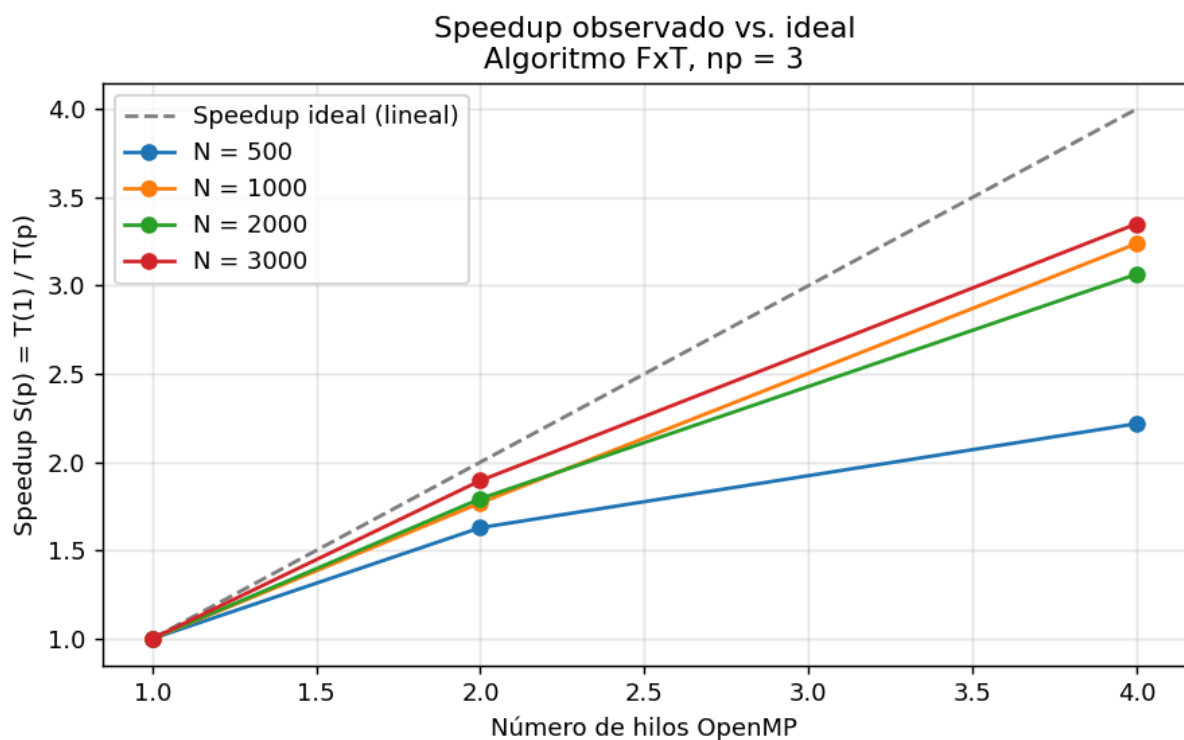


Figura 10. Speedup — FxT, np = 3.

6.3.5 Distribución de tiempos (boxplots)

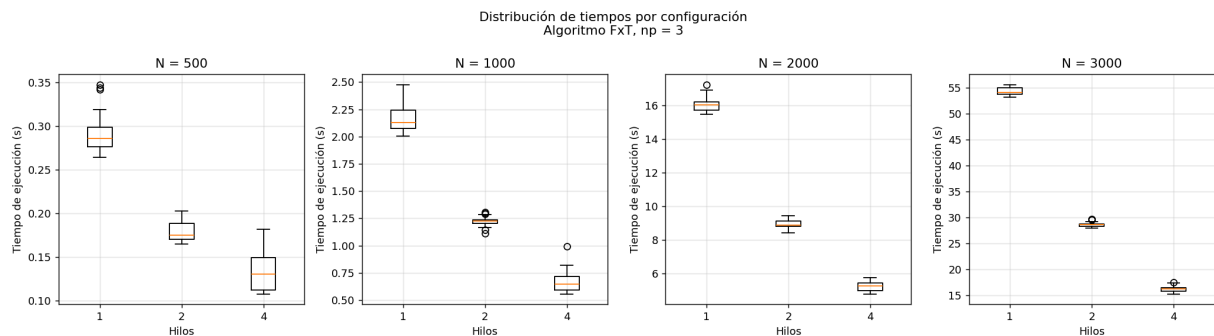


Figura 11. Distribución de las 30 repeticiones por configuración — FxT, np = 3.

Los boxplots muestran que la dispersión es muy estable a partir de $N \geq 1\,000$ (desviación relativa por debajo del 4 %). Las matrices pequeñas presentan más varianza relativa por la influencia del *jitter* del scheduler del sistema operativo y la actividad de NFS sobre tiempos absolutos cortos.

6.4 Comparación entre algoritmos FxC y FxT

La comparación pareada entre ambos algoritmos manteniendo constante `np` y los hilos permite cuantificar la ventaja del patrón de acceso optimizado de FxT.

N	Hilos	FxC media (s)	FxT media (s)	$\Delta\%$ (FxT mejor)
500	1	0.710	0.525	26.0 %
500	2	0.685	0.531	22.5 %
500	4	0.679	0.527	22.4 %
1 000	1	5.890	4.017	31.8 %
1 000	2	5.836	4.018	31.1 %
1 000	4	5.680	4.033	29.0 %
2 000	1	83.048	31.951	61.5 %
2 000	2	75.902	31.869	58.0 %
2 000	4	71.825	31.761	55.8 %
3 000	1	330.535	106.695	67.7 %
3 000	2	309.788	106.884	65.5 %
3 000	4	289.782	107.235	63.0 %

Tabla 6. Comparativa pareada FxC vs FxT manteniendo $np = 2$ constante.

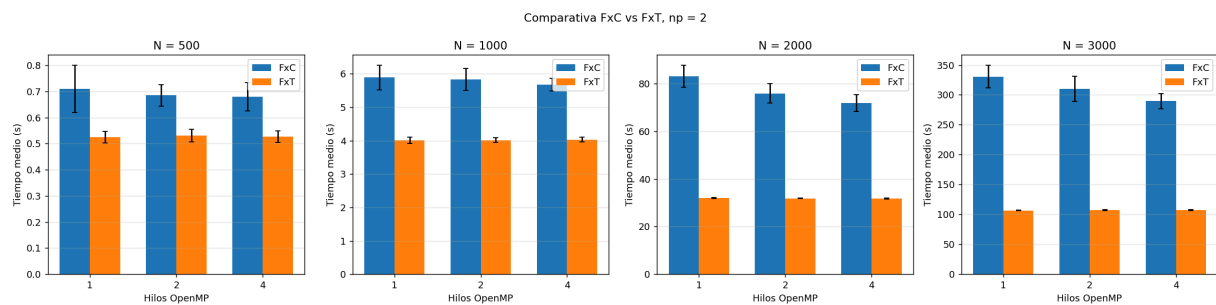


Figura 12. Comparativa de tiempos FxC vs FxT — $np = 2$.

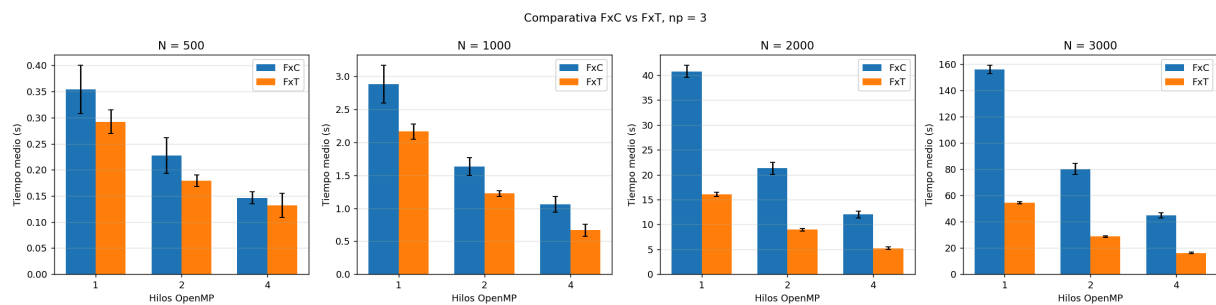


Figura 13. Comparativa de tiempos FxC vs FxT — $np = 3$.

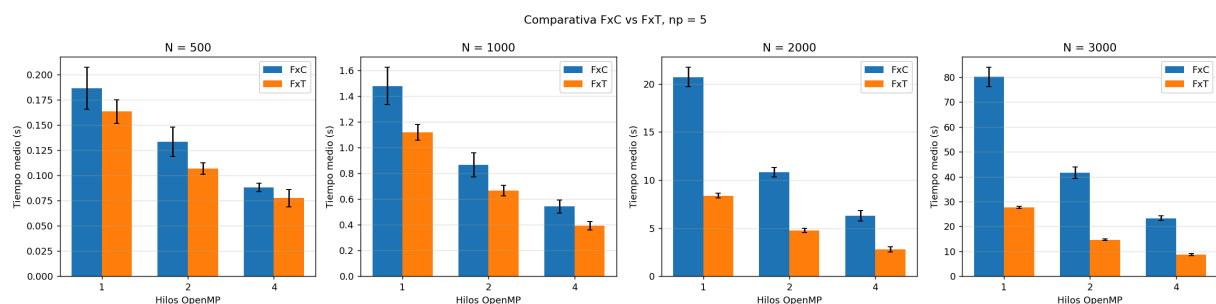


Figura 14. Comparativa de tiempos FxC vs FxT — $np = 5$.

El algoritmo FxT supera a FxC en todos los tamaños de matriz evaluados, no únicamente en los más grandes. La ventaja relativa crece con N porque el producto matricial se vuelve cada vez más *memory-bound*, y es ahí donde el patrón de acceso favorable de FxT (recorrer dos vectores contiguos) marca una diferencia más significativa. La hipótesis preliminar de que la transposición se "comería" la ganancia para matrices pequeñas no se sostuvo: dado que la transposición tiene complejidad $O(N^2)$ frente a $O(N^3)$ del producto, su costo relativo es del orden de $1/N$ y no resulta determinante incluso para $N = 500$.

6.5 Mejor configuración global por tamaño

La siguiente tabla resume la combinación óptima identificada para cada algoritmo y tamaño de matriz.

Algoritmo	N	Mejor np	Mejor hilos	Tiempo (s)	Speedup intra-nodo
FxC	500	3	4	0.146	2.42×
FxC	1 000	5	4	0.542	2.73×
FxC	2 000	3	4	11.998	3.40×

Algoritmo	N	Mejor np	Mejor hilos	Tiempo (s)	Speedup intra-nodo
FxC	3 000	3	4	44.755	3.48×
FxT	500	5	4	0.077	2.11×
FxT	1 000	5	4	0.392	2.85×
FxT	2 000	3	4	5.229	3.06×
FxT	3 000	4	4	11.063	3.30×

Tabla 7. Mejor configuración por algoritmo y tamaño de matriz.

La **mejor combinación absoluta** del taller es **FxT con N = 3 000, np = 4 y 4 hilos OpenMP**: 11.063 segundos contra 330.535 segundos del peor caso (FxC, N = 3 000, np = 2, 1 hilo) — una reducción del **96.7 %** del tiempo de pared.

7. Conclusiones y recomendaciones

7.1 Conclusiones

A partir del análisis de los datos obtenidos, se desprenden las siguientes conclusiones:

1. **El patrón de acceso a memoria es el factor de rendimiento dominante en multiplicación de matrices densas.** La variante FxT, que paga $O(N^2)$ por una transposición previa para luego recorrer dos vectores contiguos, supera consistentemente a la variante clásica FxC en todos los tamaños probados, con ventajas que oscilan entre el 22 % en N = 500 y el 68 % en N = 3 000.
2. **La distribución entre nodos físicos vía MPI es más efectiva que el paralelismo de hilos OpenMP intra-nodo en este clúster virtualizado.** Esta observación es coherente con la idea de que cada máquina virtual aporta su propio canal de memoria, mientras que los hilos OpenMP comparten el de un único nodo. Por ejemplo, para N = 3 000 el paso de np = 2 a np = 5 (de 1 a 4 workers) reduce el tiempo de 330 s a 80 s con un solo hilo, una ganancia mucho mayor que la obtenida añadiendo hilos OpenMP dentro del mismo np.
3. **El paralelismo OpenMP solo escala de manera significativa cuando hay más de un proceso MPI worker.** Con np = 2 (un único worker) los hilos compiten por la misma memoria y el speedup intra-nodo apenas alcanza 1.16×. Con np ≥ 3 el speedup se acerca al ideal lineal, alcanzando 3.48× con 4 hilos para FxC en N = 3 000 (eficiencia 0.87).
4. **A partir de N = 2 000 el escalado OpenMP es casi lineal cuando hay suficientes workers.** Las eficiencias se ubican en el rango 0.85–0.97 con 4 hilos, lo que confirma que el régimen *memory-bound* favorece la paralelización de banda.
5. **La variabilidad relativa de las mediciones decrece con N.** La desviación estándar absoluta crece con el tamaño del problema, pero la relativa baja por debajo del 4 % a

partir de $N = 2\,000$. Las 30 repeticiones por configuración son suficientes para inferencia estadística confiable.

7.2 Recomendaciones

Las observaciones anteriores se traducen en las siguientes recomendaciones prácticas:

1. **Usar siempre la variante FxT en producción.** No se identificó régimen alguno en el que FxC supere a FxT en este hardware. Si la matriz B no se puede transponer por restricciones externas (por ejemplo, si es muy grande o se modifica con frecuencia), conviene considerar variantes por bloques (*tiling*) que ofrezcan localidad sin la copia completa.
2. **Evitar invertir en hilos OpenMP cuando solo hay un worker MPI.** En la configuración degenerada `np = 2`, no resulta rentable solicitar más de un hilo: el speedup adicional no llega al 5 %.
3. **Asignar siempre el máximo de workers disponibles.** Cada worker adicional aporta un escalado prácticamente lineal hasta agotar los slots del `filehost`. La estrategia "muchos procesos $\times$ pocos hilos" supera a "pocos procesos $\times$ muchos hilos" en este clúster.
4. **Para problemas que excedan la capacidad del clúster ($N > 5\,000$)** se recomienda explorar: - Variantes por bloques (*tiling*) para mejorar la localidad de caché L2. - Bibliotecas optimizadas (BLAS, LAPACK) en lugar de implementación a mano: típicamente alcanzan más del 95 % del pico de FLOPs del hardware. - Aceleradores GPU si están disponibles, donde el speedup respecto a una CPU multinúcleo puede ser de uno o dos órdenes de magnitud.
5. **Para futuros experimentos sobre este mismo clúster** se sugiere: - Habilitar más slots por nodo en el `filehost` para explorar `np > 5`. - Medir el costo de comunicación MPI (`MPI_Bcast` y `MPI_Send/Recv`) por separado del cómputo, con el fin de aislar la fracción serial y aplicar la ley de Amdahl. - Variar el patrón de inicialización (matrices densas reales, no deterministas) para descartar efectos del compilador sobre datos predecibles.